

Learning Curriculum Builder

Claude API key provided. If you'd prefer OpenAI, ask and we'll provide one.

I The Problem

Learning something new from YouTube means choosing 4 hours of content from 400 available hours. Search and view count don't solve this. They rank what's popular, not what's right for *you*.

The signals that *would* help are scattered across video metadata, transcripts, comments, and the communities recommending this stuff. The actual hard problem is curating the right videos for a specific learner with a specific goal under a specific time budget.

I The Task

Build an AI agent that takes a learning goal, a time budget in minutes, and a user context (background, what they know, what they don't, any constraints), and produces a YouTube curriculum the learner could follow to achieve the goal within the budget.

I Reference Test Persona

Here is one persona you can develop against. We'll grade your submission against this plus additional personas you won't see and any test cases of your own.

```
{
  "persona_id": "weekend_react_dev",
  "goal": "Build a small React app this weekend, something like a habit tracker with local storage",
  "time_budget_minutes": 360,
  "user_context": {
    "background": "Python backend engineer, 5 years experience",
    "known": ["JavaScript fundamentals", "HTTP, REST, JSON", "Git, npm"],
    "unknown": ["React", "JSX", "Vite tooling", "React hooks", "Component patterns"],
    "constraints": "I prefer project-based content. Avoid pure-theory lectures and 'in 100 seconds' surface intros."
  }
}
```

I Sketch of What Good Output Looks Like

For the persona on the previous page, a ranked set of 4 to 6 videos starting with React/Vite setup, then a project-build tutorial that mirrors the habit-tracker goal, skipping JavaScript intros (already known) and surface-level "in 100 seconds" content (the persona explicitly opts out), totaling around 5 hours.

Each video should have a short reason grounded in actual content. Stronger outputs also make their reasoning legible: what was considered and dropped, how confident the agent is about each pick, and why two similar-looking videos shouldn't both make the cut.

Output format is your decision.

I Deliverables

A single GitHub repo containing:

Agent	Working code, runnable end-to-end with one command after setup. Reasonable structure: we'll read the code, not just the docs.
Test set	At least 5 input scenarios you wrote, placed in <code>/test_set/</code> . These should exercise the cases you think matter. We'll run your agent against these and against scenarios of our own.
Evaluation	The most important deliverable. How you decided whether outputs are good, what you measured, results against your test set, and what your evaluation does <i>not</i> tell you. The strongest evals also surface places where the eval and human judgment disagree, and signals you considered but discarded. Approach is your choice. We're evaluating how you think about evaluation.
README	Setup and run instructions, each significant design decision in a few sentences, and a "what I'd do with more time" section. The most compelling versions of this propose specific extensions and explain why they'd come first.

| What We Care About

- **Evaluation.** How you measure your own work. This is the deliverable we read most carefully.
- **Output quality.** Are curricula tailored to the user, or generic? Do inclusion reasons reference actual content, or just metadata?
- **Code.** Readable, sensibly structured, not over-engineered. We'd rather see clean simple code than a clever framework.
- **Design decisions.** Whether you can explain your choices.

| Bonus (Optional)

Anything from this list, or your own ideas, that you think makes the system better:

- **Follow-up Q&A.** The agent retains and defends its choices when asked, e.g. *"why didn't you include video X?"*
- **Model or prompt comparison.** Evaluate two models, or two prompt strategies; report the difference.
- **Extra discovery source.** Use a source beyond YouTube (web, Reddit, course platforms); explain why it helps.
- **Content-level deduplication.** Detect when two videos cover the same material; pick the better one.
- **Cost / latency analysis.** Measure cost-per-curriculum and latency; project what this would cost at 10K users/day and propose where you'd optimize.

| Constraints & Submission

The API key has a usage cap; if you hit it, email us and we'll extend.

Push to a public GitHub repo and send us the link when ready, with the final commit tagged `v1-submission`. Clarifying questions during the assignment are welcome.